Ciência da Computação e Sistemas de Informação · 2º Semestre

Funções, Strings e Arquivos em Python_

Introdução a Programação Estruturada em Python



MÓDULO 5

Funções — parâmetros, variáveis locais e globais, argumentos default

MÓDULO 6

Strings e Arquivos — manipulação de strings; abertura, leitura e gravação

```
def aula():
    return 'Módulos 5 e 6'
```

Professores: **Luciano · Rafael · Turatti**

Roteiro da aula

MÓDULO 5 · Funções

1. Por que funções? Anatomia de `def`
2. Parâmetros posicionais e nomeados
3. Argumentos default
4. Variáveis locais e globais
5. Exercícios rápidos

MÓDULO 6 · Strings e Arquivos

6. Indexação e fatiamento de strings
7. Métodos de manipulação
8. Formatação com f-strings
9. Abrir, ler e gravar arquivos (`open` , modos r / w / a)
10. Boas práticas com `with open`

Exercícios de fixação

Metodologia: conceito → exemplo de código → saída no console → prática guiada

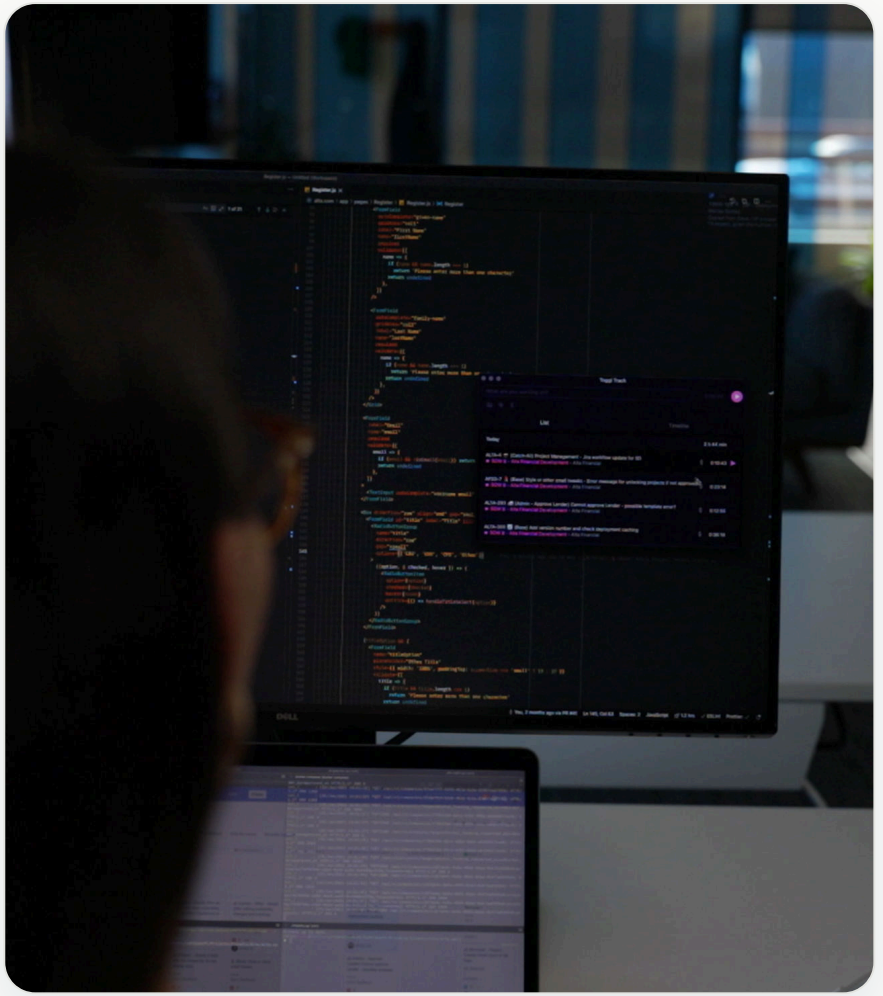
05

MÓDULO 5 · PARTE 1 DE 2

FUNÇÕES

Parâmetros · Variáveis locais e globais · Argumentos default

Objetivo: escrever blocos de código reutilizáveis — a base da programação estruturada.



Por que usar funções?

Função é um bloco de código com nome que executa uma tarefa e pode ser chamado quantas vezes forem necessárias.

Reuso — escreva uma vez, use em todo o programa

Decomposição — problemas grandes viram partes pequenas e testáveis

Abstração — quem chama precisa saber O QUÊ faz, não COMO

Manutenção — corrigiu em um lugar, corrigiu em todos

```python
def boas_vindas(nome):
    print(f'Boa noite, {nome}!')

boas_vindas('Maria')   # Boa noite, Maria!
boas_vindas('João')    # Boa noite, João!
boas_vindas('Turma')   # Boa noite, Turma!
```

Sem função, as três chamadas virariam três `print` copiados à mão — e qualquer mudança seria trabalho triplo.

Anatomia de uma função

```python
def calcula_media(n1, n2):
    soma = n1 + n2
    media = soma / 2
    return media

nota = calcula_media(8.0, 6.5)
print(nota)                    # 7.25
```

1. `def` + nome + parâmetros entre parênteses
2. corpo indentado (4 espaços)
3. `return` devolve o resultado e encerra
4. chamada com argumentos reais

Regras que valem ouro

- `def` cria a função — nada roda até a chamada
- Sem `return`, a função devolve `None`
- Nomes em `minúsculas_com_underline` (snake_case)
- Uma função = uma tarefa clara

Parâmetros: posicionais e nomeados

Parâmetro é a variável declarada no `def`; **argumento** é o valor real enviado na chamada.

Posicionais — a ordem importa

```
def apresenta(nome, idade):
    print(f'{nome} tem {idade} anos')

apresenta('Ana', 20)    # Ana tem 20 anos
apresenta(20, 'Ana')    # 20 tem Ana anos — trocou, quebrou!
```

Nomeados — a ordem não importa

```
apresenta(idade=20, nome='Ana')
# Ana tem 20 anos

# prática comum: chamada autodocumentada
apresenta(nome='Ana', idade=20)
```

Vantagem dos nomeados: chamadas legíveis e possibilidade de omitir parâmetros que têm `default` (próximo slide).

Argumentos default (valor padrão)

Parâmetro com valor padrão torna o argumento opcional: quem chama pode omitir.

Regras

1. Obrigatórios primeiro, defaults por **ÚLTIMO**: `def pergunta(texto, tentativas=3)`
2. O default é avaliado **na definição** da função, uma única vez
3. Ideal para valores frequentes — reduz chamadas repetidas

```
def saudacao(nome='mundo', hora='dia'):
    print(f'Boa {hora}, {nome}!')

saudacao()                  # Boa dia, mundo!
saudacao('Maria')           # Boa dia, Maria!
saudacao(hora='noite')      # Boa noite, mundo!
saudacao('João', 'tarde')   # Boa tarde, João!
```

Armadilha clássica: default mutável

Não faça assim

```
def adiciona(tarefa, lista=[]):  # criada UMA única vez
    lista.append(tarefa)
    return lista

print(adiciona('estudar'))   # ['estudar']
print(adiciona('dormir'))    # ['estudar', 'dormir']
# a lista VAZOU entre as chamadas!
```

Faça assim

```
def adiciona(tarefa, lista=None):
    if lista is None:
        lista = []  # nova lista a cada chamada
    lista.append(tarefa)
    return lista
```

Nunca use `list` ou `dict` como default. Use `None` e crie a estrutura dentro da função.

MÓDULO 5 · FUNÇÕES · ESCOPO

Variáveis locais

Variável criada dentro de uma função **nasce na chamada e morre no return** — existe só lá dentro.

escopo_local.py

```
def calcula_desconto(valor):
    taxa = 0.10                    # LOCAL à função
    desconto = valor * taxa
    return desconto

print(calcula_desconto(200))       # 20.0
print(taxa)                        # NameError: name 'taxa' is not def
```

CHAMADA a variável nasce → **EXECUÇÃO** ela existe → **RETURN** ela morre

Por que isso é bom?

- Cada função pode usar os **mesmos nomes** sem brigar entre si
- Evita **efeitos colaterais**: a função não mexe nas variáveis de fora
- Facilita **testar**: basta passar argumentos e conferir o return

Mini desafio: se chamar `calcula_desconto(400)`, quanto vale `taxa` durante a chamada?

MÓDULO 5 · FUNÇÕES · ESCOPO

Variáveis globais e a palavra-chave global

Variável definida fora de qualquer função é global: o programa inteiro enxerga. Dentro da função, **LER é livre** — **ALTERAR exige global**.

LER é livre

A função enxerga e usa a global normalmente — nenhuma palavra-chave necessária.

ALTERAR exige global

Atribuir sem a keyword cria uma variável local nova e o Python dispara `UnboundLocalError`.

globals.py

```
saldo = 1000                    # GLOBAL

def saque(valor):
    global saldo                # permissão para modificar
    saldo = saldo - valor

saque(200)
```

LEITURA SEM KEYWORD

constantes.py

```
TAXA_IR = 0.15              # constante global (maiúsculas)

def liquido(salario):
    return salario * (1 - TAXA_IR)   # ler global: sem keyword
```

MÓDULO 5 · FUNÇÕES · ESCOPO

Escopo: local vs global

Aspecto	Local	Global
Onde nasce	Dentro da função	Fora de todas as funções
Ciclo de vida	Recriada a cada chamada	Vive até o fim do programa
Visibilidade	Somente na própria função	Em todo o arquivo
Alterar dentro de função	Direto	Precisa da palavra-chave `global`

Boas práticas

- Prefira **entrada por parâmetros** e **saída por return**
- Reserve globals para **constantes e configurações** `TAXA_IR = 0.15`
- Função que altera o mundo lá fora é **difícil de testar e depurar**

Mão na massa — Módulo 5

10–15 min · acompanhem no REPL

01

Função média

Escreva `media(n1, n2, n3)` que retorna a média aritmética das três notas. Teste com 7, 8, 9 → 8.0.

```
print(media(7, 8, 9)) → 8.0
```

02

Boleto com default

Escreva `boleto(valor, dias_atraso=0)` que soma 2% de multa por dia de atraso. Chame com e sem o segundo argumento.

```
boleto(100) # sem atraso
boleto(100, 3) # com atraso
```

03

Caça ao escopo

```
x = 5
def troca():
    x = 10
troca()
print(x)
```

Qual a saída?

MÓDULO 6 · PARTE 2 DE 2

06

STRINGS E ARQUIVOS

Manipulação de strings · Abertura, leitura e gravação em arquivos

Objetivo: processar texto de verdade — da string na memória ao arquivo no disco.



MÓDULO 6 · STRINGS

Strings: indexação e fatiamento

A PALAVRA "PYTHON", CARACTERE POR CARACTERE

P	y	t	h	o	n
0	1	2	3	4	5
-6	-5	-4	-3	-2	-1

[início : fim : passo]

O fatiamento **inclui o início** e **exclui o fim** — `frase[1:4]` devolve 'yth'.

strings_fatiar.py

```
frase = 'Python'

frase[0]     # 'P'   primeiro caractere
frase[-1]    # 'n'   negativo conta do fim
frase[1:4]   # 'yth' inclui o início, exclui o fim
frase[2:]    # 'thon'
frase[:3]    # 'Pyt'
frase[::-1]  # 'nohtyP' passo negativo inverte
```

Strings são IMUTÁVEIS: `frase[0] = "J"` gera `TypeError`. Os métodos criam strings novas.

Métodos essenciais de manipulação

BASE `s = '  Programação em Python  '`

`s.strip()`

`"Programação em Python"` — remove espaços das pontas

`s.upper()`

`"  PROGRAMAÇÃO EM PYTHON  "`

`s.lower()`

`"  programação em python  "`

`s.replace("em", "com")`

`"  Programação com Python  "`

`s.split()`

`['Programação', 'em', 'Python']` — quebra em lista

`"-".join(["a", "b", "c"])`

`"a-b-c"` — cola a lista em string

`"py" in s.lower()`

`True` — operador de pertinência

`s.count("a") / len(s)`

`3 / 26` — contagem e tamanho

Nenhum método altera `s` — todos devolvem strings novas. Combinação clássica para limpar entrada de usuário: `entrada.strip().lower()`

Formatação com f-strings

Coloque `f` antes das aspas e insira variáveis ou expressões dentro de `{ }`.

fstrings_demo.py

```python
nome = 'Ana'
nota = 8.4567

print(f'Aluna: {nome}')           # Aluna: Ana
print(f'Nota: {nota:.1f}')        # Nota: 8.5
print(f'Dobro: {nota * 2:.2f}')   # Dobro: 16.91
print(f'Aluna {nome} tirou {nota:.1f} em Python')
```

Dicas rápidas

- `:.1f` — arredonda para 1 casa decimal
- `:>10` — alinha à direita em 10 colunas
- Aceita qualquer expressão: `{2 * 3}`, `{len(nome)}`
- Padrão moderno (Python 3.6+) — prefira a `%` e `.format()`

Arquivos: `open()` e modos de abertura

```
arquivo = open('notas.txt', 'r')   # (caminho, modo)
# ... usar o arquivo ...
arquivo.close()                    # libera o recurso
```

⚠ Modo `w` apaga sem aviso — confira antes de abrir!

MODO	SIGNIFICADO	SE NÃO EXISTE	SE EXISTE
`'r'`	leitura (padrão)	erro `FileNotFoundError`	lê desde o início
`'w'`	escrita (sobrescreve)	cria novo	**APAGA o conteúdo atual**
`'a'`	anexar no final (append)	cria novo	mantém e escreve no fim
`'r+'`	leitura e escrita	erro	preserva o conteúdo

Lendo arquivos

TÉCNICA 01

tudo de uma vez

```
f = open('alunos.txt', 'r')
texto = f.read()   # todo o texto como uma string
f.close()
```

TÉCNICA 02

linha por linha

```
linha = f.readline()  # traz o \n no fim → use .strip()
```

TÉCNICA 03

iterando (recomendado)

```
with open('alunos.txt', 'r') as f:
    for linha in f:
        print(linha.strip())
```

alunos.txt contém Ana, Bia, Caio → o **for** imprime Ana, Bia, Caio, uma por linha.

for linha in f lê sob demanda: funciona até em arquivos gigantes.

Gravando em arquivos

`open(...) as f` + `f.write(texto)`. O modo decide substituir ou acrescentar.

gravar_log.py

```
with open('log.txt', 'w') as f:      # cria / zera o arquivo
    f.write('Aula de Python\n')
    f.write('Módulo 6: arquivos\n')

with open('log.txt', 'a') as f:      # acrescenta no final
    f.write('Fim da aula\n')
```

log.txt depois da execução

```
Aula de Python
Módulo 6: arquivos
Fim da aula
```

↵ `write()` NÃO pula linha: inclua o `\n` no final

⇄ `w` zera o arquivo; `a` preserva

<> Grave sempre texto (`str`) — converta números com f-string ou `str()`

Boas práticas: `with open`

O gerenciador de contexto fecha o arquivo automaticamente — mesmo se um erro acontecer no meio do bloco.

Checklist do arquivista

- Use `with open` para TODA leitura e gravação
- Só use `open()` puro se o arquivo precisar ficar aberto por muito tempo
- Em arquivos grandes, leia linha a linha dentro do `with`
- Verifique existência: `import os` → `os.path.exists(caminho)`

entrada.txt → saida.txt

```
with open('entrada.txt', 'r') as origem, \
     open('saida.txt', 'w') as destino:
    for linha in origem:
        destino.write(linha.upper())
# nada de .close(): o with cuida de tudo
```

MÓDULO 6 · STRINGS E ARQUIVOS

Mão na massa — Módulo 6

15–20 min · em duplas

6

EXERCÍCIO 1

Contador de vogais

Leia uma frase do teclado e mostre quantas vogais ela tem. Dica: `frase.lower().count("a")` para cada vogal, ou um `for` + `in`.

EXERCÍCIO 2

Cadastro em arquivo

Peça 3 nomes ao usuário e grave um por linha em `cadastro.txt` (modo `w`).

EXERCÍCIO 3

Relatório lido

Reabra `cadastro.txt` em modo `r` e mostre os nomes em MAIÚSCULAS e numerados: `1. ANA`

Os três exercícios combinam strings do Módulo 6 com arquivos — encadeie-os na mesma sessão do interpretador.

Exercícios de revisão

Um desafio por tópico: os dois primeiros revisitam o Módulo 5, os dois últimos o Módulo 6.

4 desafios · em duplas · ~20 min

MÓDULO 5

01 Maior de três

Escreva `maior_de_tres(a, b, c)` que retorna o maior dos três valores. Dica: assuma que `a` é o maior e vá atualizando a variável.

```
maior_de_tres(7, 12, 9) → 12
```

MÓDULO 5

02 Ficha com default

Escreva `ficha(nome, curso='Python', nivel=1)` que retorna a frase **Nome cursa X — nível Y**. Faça três chamadas: só nome; nome + curso; nivel nomeado.

```
ficha('Caio', nivel=3)
→ Caio cursa Python — nível 3
```

MÓDULO 6

03 Contador de vogais

Escreva `conta_vogais(frase)` que retorna quantas vogais a frase tem. Dica: percorra `frase.lower()` e teste `if letra in 'aeiou'`.

```
conta_vogais('Python para todos') → 5
```

MÓDULO 6

04 Lista para o arquivo

Escreva `salvar_linhas(caminho, itens)` que grava cada item em uma linha do arquivo (modo `w`) e retorna a quantidade de linhas gravadas. Teste com três cores.

```
salvar_linhas('cores.txt', cores) → 3
```

Resolução comentada

compare com a sua versão

Σ 01 · Maior de três

MÓDULO 5

```python
def maior_de_tres(a, b, c):
    maior = a
    if b > maior:
        maior = b
    if c > maior:
        maior = c
    return maior
# alternativa: return max(a, b, c)
```

02 · Ficha com default

MÓDULO 5

```python
def ficha(nome, curso='Python', nivel=1):
    return f'{nome} cursa {curso} — nível {nivel}'

ficha('Ana')            # Ana cursa Python — nível 1
ficha('Bia', 'Java')    # Bia cursa Java — nível 1
ficha('Caio', nivel=3)  # Caio cursa Python — nível 3
```

Tт 03 · Contador de vogais

MÓDULO 6

```python
def conta_vogais(frase):
    total = 0
    for letra in frase.lower():
        if letra in 'aeiou':
            total += 1
    return total

conta_vogais('Python para todos')   # 5
```

04 · Lista para o arquivo

MÓDULO 6

```python
def salvar_linhas(caminho, itens):
    with open(caminho, 'w') as f:
        for item in itens:
            f.write(item + '\n')
    return len(itens)

salvar_linhas('cores.txt', cores)   # 3 linhas gravadas
```

ENCERRAMENTO · MÓDULOS 5 & 6

O que levamos hoje

Σ MÓDULO 5 — Funções

- ✓ `def`, parâmetros posicionais/nomeados e defaults tornam o código reutilizável
- ✓ Escopo: local nasce e morre na função; global exige a palavra-chave `global` para alterar

🗀 MÓDULO 6 — Strings e Arquivos

- ✓ Strings são imutáveis; fatiamento `[início:fim:passo]` e métodos geram strings novas
- ✓ `with open` + modos `r`, `w`, `a` cobrem leitura e gravação com segurança

Dúvidas? Procurem os professores Luciano, Rafael e Turatti.

